SLOT 7

USER INTERACTION, THEME & NAVIGATION

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

- Pickers – Date/Time
- Theming & UI Polishing
- Navigator 1.0 – Push/Pop

Mục tiêu tổng quát của Slot 7

👉 Giúp sinh viên:

- Biết **tương tác đúng cách với người dùng**
- Làm UI **đồng bộ – đẹp – dễ nhìn hơn**
- **Chuyển màn hình đúng chuẩn** như một ứng dụng Flutter thực tế

1. Pickers – Date & Time

Khái niệm cốt lõi

- **Picker** là **widget/hộp thoại hỗ trợ người dùng chọn dữ liệu**, thay vì nhập tay
- `DatePicker` & `TimePicker` dùng để **chọn ngày / giờ một cách trực quan**
- Picker **không phải là TextField thường**, mà là **dialog tương tác**

Hiểu đúng: Picker giúp **giảm lỗi nhập liệu** và **tăng trải nghiệm người dùng**

Mục tiêu học tập

Sau phần này, sinh viên có thể:

- Mở **DatePicker / TimePicker** khi người dùng tap
- Lấy giá trị ngày/giờ được chọn và **hiển thị lại trên UI**
- Áp dụng picker vào **form thực tế** (đặt lịch, đăng ký, deadline…)

Mô tả ngắn (ý nghĩa & vai trò)

- Giúp sinh viên hiểu **tương tác người dùng là sự kiện + phản hồi UI**
- Là nền tảng cho các form thật trong app:
  - 👉 đặt lịch hẹn, chọn ngày sinh, thời gian học, nhắc việc

2. Theming & UI Polishing

Khái niệm cốt lõi

- **Theme** là cách định nghĩa **màu sắc – font – style dùng chung cho toàn app**
- Flutter cho phép:
  - Theme toàn app (`ThemeData`)
  - Theme sáng / tối (Light – Dark)
- UI Polishing = **chỉnh giao diện cho dễ nhìn, nhất quán, không rối**

Hiểu đúng: Theme **không phải trang trí cho đẹp**, mà để **app có phong cách thống nhất**

Mục tiêu học tập

Sau phần này, sinh viên có thể:

- Định nghĩa **màu chủ đạo** cho app
- Áp dụng theme cho:
  - AppBar
  - Button
  - Text
- Tránh UI "mỗi màn hình một kiểu"

PhucPT10@fpt.edu.vn

📝 Mô tả ngắn (ý nghĩa & vai trò)

- Giúp sinh viên chuyển từ:
  👉 "code chạy được" → "app nhìn giống sản phẩm thật"
- Là bước đầu tiếp cận **UI/UX cơ bản** trong Flutter

🧱 3. Navigator 1.0 – Push / Pop

🔑 Khái niệm cốt lõi

- **Navigator** quản lý việc **chuyển màn hình trong Flutter**
- `push()` → mở màn hình mới
- `pop()` → quay lại màn hình trước
- Các màn hình được quản lý theo **ngăn xếp (stack)**

Hiểu đúng: Điều hướng **không phải đổi widget**, mà là **điều khiển luồng màn hình**

🎯 Mục tiêu học tập

Sau phần này, sinh viên có thể:

- Chuyển từ màn hình danh sách → màn hình chi tiết
- Truyền dữ liệu khi chuyển màn hình
- Quay lại màn hình trước đúng cách

📝 Mô tả ngắn (ý nghĩa & vai trò)

- Là **xương sống của mọi app Flutter nhiều màn hình**
- Giúp sinh viên xây dựng:
  👉 app danh sách – chi tiết
  👉 app đăng nhập – trang chính
  👉 app nhiều luồng sử dụng

🎓 KẾT LUẬN SƯ PHẠM – SLOT 7 GIÚP GÌ CHO SINH VIÊN?

Sau Slot 7, sinh viên **có thể tự tin trả lời**:

✅ *"Tôi biết cách cho người dùng chọn ngày giờ đúng cách"*
✅ *"Tôi biết làm app có màu sắc thống nhất, nhìn dễ chịu hơn"*
✅ *"Tôi biết chuyển màn hình trong Flutter giống một app thật"*
👉 Đây là **ranh giới rõ ràng giữa bài tập Flutter và ứng dụng Flutter thực tế**

📚 TÀI LIỆU THAM KHẢO UY TÍN

1. **Flutter Official Docs – UI & Navigation**
   https://docs.flutter.dev/ui
   https://docs.flutter.dev/ui/navigation
2. **Flutter Date & Time Pickers**
   https://api.flutter.dev/flutter/material/showDatePicker.html
3. **Flutter Theming Guide (Official)**
   https://docs.flutter.dev/cookbook/design/themes
4. **Flutter Navigator 1.0 – Easy Explanation (YouTube)**
   Tìm: *"Flutter Navigator push pop for beginners"*
5. **Flutter Cookbook – Practical Examples**
   https://docs.flutter.dev/cookbook

PhucPT10@fpt.edu.vn

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1. Pickers – Date & Time

Khái niệm quan trọng (PHẢI HIỂU ĐÚNG)

- **Picker là cơ chế tương tác**, không phải widget hiển thị trực tiếp trên màn hình
- `showDatePicker` / `showTimePicker`:
  - mở **dialog**
  - trả về **Future** (kết quả chọn hoặc null)
- Picker dùng để **giảm nhập tay**, **giảm lỗi**, **tăng UX**

👉 Sinh viên **hay hiểu sai**:

- Nghĩ picker là `TextField`
- Nghĩ gọi picker là code chạy ngay, **không cần async/await**
- Không hiểu vì sao picker **không mở**

Kỹ thuật / thao tác PHẢI THÀNH THẠO

- Gọi:
  - `showDatePicker(...)`
  - `showTimePicker(...)`
- Xử lý kết quả:
  - dùng `async / await`
  - kiểm tra `null`
- Hiển thị lại dữ liệu đã chọn lên UI (Text, Button, Form)
- Gắn picker vào **sự kiện người dùng** (`onTap`, `onPressed`)

👉 Mức yêu cầu: **làm được picker trong form thật**, không chỉ chạy demo

Lỗi thường gặp (kèm nguyên nhân)

- ❌ Picker không mở
  → dùng `context` không hợp lệ (gọi sau dispose, sai cấp widget)
- ❌ App crash khi chọn xong
  → không kiểm tra `null`
- ❌ Chọn rồi nhưng UI không đổi
  → quên `setState`
- ❌ Logic xử lý nhét hết vào UI
  → không tách phần xử lý dữ liệu

Best Practices (nên áp dụng sớm)

- Tách:
  - **UI mở picker**
  - **logic xử lý dữ liệu**
- Luôn kiểm tra `null` khi picker đóng
- Không hard-code format ngày giờ → dùng format rõ ràng
- Picker chỉ dùng khi **người dùng chủ động tương tác**

Ý nghĩa sư phạm

Picker giúp sinh viên hiểu:

👉 **UI tốt là giúp người dùng chọn đúng – dễ – ít sai**,
rất quan trọng trong **form nhập liệu, đặt lịch, đăng ký, deadline** của app thật.

2. Theming & UI Polishing

Khái niệm quan trọng (PHẢI HIỂU ĐÚNG)

- **Theme là cấu hình giao diện dùng chung cho toàn app**

PhucPT10@fpt.edu.vn

- Theme không chỉ là màu → gồm:
  - màu sắc
  - font chữ
  - style button, text
- UI Polishing = **làm giao diện dễ nhìn, nhất quán, không rối**

👉 Sinh viên **hay hiểu sai**:
- Mỗi widget tự set màu cho nhanh
- Nghĩ Theme chỉ để "làm đẹp"
- Không phân biệt theme **toàn app** và **cục bộ**

◈ Kỹ thuật / thao tác PHẢI THÀNH THẠO
- Khai báo `ThemeData` trong `MaterialApp`
- Dùng:
  - `Theme.of(context)`
  - `ColorScheme`
- Áp dụng theme cho:
  - AppBar
  - Button
  - Text
- Biết **chỉnh spacing, padding, font size cơ bản**

👉 Mức yêu cầu: **app nhìn nhất quán**, không cần quá đẹp

◈ Lỗi thường gặp (kèm nguyên nhân)
- ❌ UI không đổi theme
  → đặt `ThemeData` sai cấp
- ❌ Giao diện rối, khó nhìn
  → thiếu spacing, màu tương phản kém
- ❌ Mỗi màn hình một style
  → không dùng theme chung
- ❌ Code UI dài, khó đọc
  → hard-code màu, font everywhere

◈ Best Practices (nên áp dụng sớm)
- Luôn ưu tiên **theme > hard-code**
- Giữ số màu chủ đạo **ít (2–3 màu)**
- Text size rõ ràng: tiêu đề – nội dung – phụ
- UI đơn giản > màu mè

📌 Ý nghĩa sư phạm
Theming giúp sinh viên chuyển tư duy từ:

👉 *"code cho chạy" → "làm app có phong cách, giống sản phẩm"*

Rất quan trọng với **trải nghiệm người dùng và dự án nhóm**.

▨ 3. Navigator 1.0 – Push / Pop
◈ Khái niệm quan trọng (PHẢI HIỂU ĐÚNG)
- **Navigator quản lý luồng màn hình bằng stack**
- `push` = thêm màn hình mới lên stack
- `pop` = quay lại màn hình trước
- Điều hướng ≠ đổi widget trong cùng màn hình

👉 Sinh viên **hay hiểu sai**:
- Dùng `push` cho mọi trường hợp

PhucPT10@fpt.edu.vn

- Không hiểu vì sao `pop` không quay lại
- Nhầm giữa **UI state** và **navigation state**

◈ Kỹ thuật / thao tác PHẢI THÀNH THẠO

- Dùng:
  - `Navigator.push`
  - `Navigator.pop`
- Truyền dữ liệu:
  - khi `push`
  - khi `pop` (mức cơ bản)
- Điều hướng:
  - danh sách → chi tiết
  - form → màn xác nhận

👉 Mức yêu cầu: **xây được app nhiều màn hình đơn giản**

◈ Lỗi thường gặp (kèm nguyên nhân)

- ❌ Pop không quay về màn trước
  → push sai luồng, push chồng không kiểm soát
- ❌ Truyền dữ liệu lỗi
  → truyền sai kiểu, context không đúng
- ❌ Điều hướng rối
  → không vẽ trước luồng màn hình
- ❌ Logic điều hướng nằm rải rác
  → gọi Navigator lung tung trong UI

◈ Best Practices (nên áp dụng sớm)

- Nghĩ **luồng màn hình trước khi code**
- Push rõ ràng, pop đúng chỗ
- Truyền dữ liệu tối thiểu, rõ ràng
- Không lồng push phức tạp ở slot này

📌 Ý nghĩa sư phạm

Navigator là **xương sống của app nhiều màn hình**.
Nắm vững Navigator 1.0 giúp sinh viên làm được:

👉 app danh sách – chi tiết
👉 app đăng nhập – trang chính
👉 luồng sử dụng giống app thật

🎯 TỔNG KẾT MỤC TIÊU SƯ PHẠM – SLOT 7

Sau Slot 7, sinh viên **PHẢI**:

- ✅ Hiểu đúng bản chất:
  - Picker
  - Theme
  - Navigation
- ✅ Làm được:
  - form chọn ngày giờ
  - app có giao diện thống nhất
  - chuyển màn hình đúng cách
- 🔍 Phân biệt được:
  - **cốt lõi bắt buộc** (slot này)
  - **nâng cao** (chưa cần vội)

PhucPT10@fpt.edu.vn

III. ỨNG DỤNG THỰC TIỄN

1 VAI TRÒ CỦA SLOT 7 TRONG PHÁT TRIỂN APP FLUTTER THỰC TẾ

◈ Slot 7 được dùng ở đâu trong app thật?

Trong **gần như mọi ứng dụng Flutter**, Slot 7 xuất hiện ở:

- 📝 **Form nhập liệu**
  - chọn ngày sinh
  - chọn deadline công việc
  - chọn giờ nhắc việc
- ✏️ **Màn hình tạo / chỉnh sửa dữ liệu**
  - thêm task
  - sửa lịch
- 🧭 **Luồng thao tác người dùng**
  - danh sách → chi tiết
  - form → xác nhận → quay lại

◈ Vai trò của từng nhóm kỹ thuật

🎯 *User Interaction – Pickers*

- Giúp người dùng **chọn đúng dữ liệu**
- Tránh:
  - nhập sai format
  - nhập nhầm ngày / giờ
- Là yếu tố UX **cực kỳ quan trọng** trong app thật

🎨 *Theming & UI Polishing*

- Tạo **cảm giác app chuyên nghiệp**
- Giữ UI:
  - nhất quán
  - dễ nhìn
  - dễ sử dụng
- Rất quan trọng với:
  - MVP
  - app nội bộ
  - app sinh viên làm đồ án

🧭 *Navigation – Navigator 1.0*

- Kiểm soát **luồng sử dụng**
- Giúp người dùng:
  - biết mình đang ở đâu
  - quay lại đúng chỗ
- Là nền tảng của **mọi app nhiều màn hình**

👉 **Thông điệp sư phạm cốt lõi**

Slot 7 là bước chuyển từ:

❌ *"app demo chạy được"*

✅ *"app giống sản phẩm thật"*

2 CÁC TÌNH HUỐNG THỰC TẾ (3 TÌNH HUỐNG)

▨ Tình huống 1 – App quản lý công việc cá nhân (Todo App)

◈ Bối cảnh

- App cho người dùng:

PhucPT10@fpt.edu.vn

    - tạo công việc
    - đặt deadline
    - xem chi tiết task

Vấn đề gặp phải

- Người dùng nhập deadline bằng TextField → **sai ngày**
- UI màn tạo task và màn chi tiết **không giống nhau**
- Chuyển màn hình rối, khó quay lại

Áp dụng kỹ thuật Slot 7

- ✅ **Pickers**
    - dùng `showDatePicker` chọn deadline
    - hiển thị ngày đã chọn trên form
- ✅ **Theme**
    - dùng `ThemeData` cho:
        - AppBar
        - Button
        - Text
- ✅ **Navigator 1.0**
    - `Navigator.push` từ list → detail
    - `Navigator.pop` sau khi tạo task xong

Mục tiêu đạt được

- UX tốt hơn → người dùng **ít sai**
- UI nhìn **giống app thật**
- Luồng sử dụng **rõ ràng – dễ hiểu**

Tình huống 2 – App lịch cá nhân / nhắc việc

Bối cảnh

- App cho phép:
    - tạo lịch hẹn
    - đặt giờ nhắc
    - chỉnh sửa lịch

Vấn đề gặp phải

- Nhập giờ thủ công → **nhầm AM / PM**
- UI mỗi màn hình một kiểu
- Khi quay lại màn trước → **mất dữ liệu**

Áp dụng kỹ thuật Slot 7

- ✅ **showTimePicker**
    - chọn giờ trực quan
    - xử lý kết quả `async / await`
- ✅ **Theme.of(context)**
    - dùng chung màu, font
- ✅ **Navigator.pop(context, result)**
    - trả dữ liệu về màn trước

Mục tiêu đạt được

- Trải nghiệm người dùng **tự nhiên**
- App dùng **thoải mái, không gây khó chịu**

PhucPT10@fpt.edu.vn

- Sinh viên hiểu:
  "Navigation không chỉ là đổi màn hình"

Tình huống 3 – App sinh viên / app nội bộ công ty

Bối cảnh

- App có:
  - danh sách sinh viên / nhân viên
  - màn hình chi tiết
  - form chỉnh sửa thông tin

Vấn đề gặp phải

- Điều hướng lộn xộn
- UI nhìn "học sinh"
- Code UI dài, khó đọc

Áp dụng kỹ thuật Slot 7

- ✅ **Navigator 1.0**
  - luồng rõ: list → detail → edit
- ✅ **Theme**
  - bỏ hard-code màu
  - UI nhất quán
- ✅ **Pickers**
  - chọn ngày sinh / ngày vào làm

Mục tiêu đạt được

- App **giống sản phẩm nội bộ thật**
- Dễ mở rộng về sau
- Sinh viên **tự tin làm đồ án**

3 CASE STUDY – ÁP DỤNG SLOT 7 (MÔ PHỎNG)

Bối cảnh dự án

- MVP app quản lý công việc cho nhóm nhỏ (3–5 người)

Vấn đề ban đầu

- UX kém:
  - nhập ngày giờ rối
- UI:
  - mỗi màn hình một kiểu
- Navigation:
  - người dùng không biết đang ở đâu

Cách áp dụng Slot 7

- **Pickers**
  - chọn deadline, giờ nhắc
- **Theme**
  - 1 màu chủ đạo
  - font & spacing thống nhất
- **Navigator 1.0**
  - kiểm soát luồng:
    - Home → Add Task → Detail → Back

PhucPT10@fpt.edu.vn

✅ Kết quả

- App:
  - dễ dùng hơn
  - nhìn chuyên nghiệp
- Sinh viên:
  - hiểu *vì sao phải học Slot 7*
  - không còn hỏi: "học cái này để làm gì?"

4 BÀI HỌC RÚT RA (LESSONS LEARNED)

🔑 Điều quan trọng nhất

- **Kỹ thuật chỉ có giá trị khi giải quyết được vấn đề người dùng**

❌ Sai lầm phổ biến của sinh viên

- Hard-code màu → UI rối
- Lạm dụng `Navigator.push`
- Dùng picker nhưng xử lý dữ liệu sai
- Không nghĩ luồng màn hình trước khi code

✅ Nguyên tắc cần duy trì lâu dài

- Luôn nghĩ:
  "Người dùng thao tác thế nào?"
- UI:
  - đơn giản
  - nhất quán
- Navigation:
  - rõ ràng
  - dễ quay lại
- Code:
  - dễ đọc
  - dễ sửa

PhucPT10@fpt.edu.vn

IV. MỘT SỐ NỘI DUNG CHI TIẾT

◈ PHẦN A. PICKERS – DATE & TIME

(User Input chuẩn UX trong Flutter)

I. TƯ DUY NHẬP LIỆU TRONG ỨNG DỤNG THỰC TẾ

❓ Vì sao KHÔNG nên cho người dùng nhập ngày/giờ bằng TextField?

🔑 *Tư duy cốt lõi*

- Ngày & giờ là **dữ liệu có cấu trúc**
- Con người **dễ nhập sai** hơn máy
- UX tốt = **giảm tối đa việc phải nhớ & gõ**

⚠️ *Rủi ro khi dùng TextField*

- ❌ Sai định dạng
  `12/1/24` – `01-12-2024` – `2024/12/01`
- ❌ Khó validate
  → code phức tạp, dễ bug
- ❌ UX kém
  → người dùng bối rối, nhập chậm

📱 *Liên hệ app thật*

- App công việc → deadline
- App booking → ngày đặt lịch
- App sinh viên → ngày sinh

👉 **Tất cả app thật đều dùng picker**, không cho nhập tay

❌ *Sinh viên hay hiểu sai*

- "TextField nhập nhanh hơn"
- "Validate là xong"
- "Picker chỉ để cho đẹp"

➡️ Sai bản chất: **Picker là giải pháp UX, không phải UI decoration**

II. DATE PICKER TRONG FLUTTER

◈ showDatePicker là gì?

- Là **hàm mở dialog chọn ngày**
- Không phải widget hiển thị cố định
- Trả về `Future<DateTime?>`

🔁 Luồng hoạt động chuẩn

1. Người dùng tap nút / ô nhập
2. `showDatePicker` mở dialog
3. Người dùng chọn ngày hoặc cancel
4. Hàm trả về:
   - `DateTime` (nếu chọn)
   - `null` (nếu hủy)

🔑 Tham số quan trọng

- `initialDate`
  → ngày hiển thị ban đầu
- `firstDate`
  → ngày nhỏ nhất được chọn

PhucPT10@fpt.edu.vn

- `lastDate`
  → ngày lớn nhất được chọn

👉 Flutter **bắt buộc** các tham số này để:
- tránh chọn ngày vô lý
- hỗ trợ validate từ UI

🧠 Vì sao Flutter thiết kế `async`?
- Picker là **tương tác người dùng**
- Không biết khi nào người dùng chọn xong
- Flutter **không block UI**

👉 `async/await` = chờ **hành động con người**

❗ Vì sao phải xử lý `null`?
- Người dùng có quyền:
  - đóng dialog
  - không chọn gì
- Không xử lý `null` → **crash app**

❌ Lỗi phổ biến
- Không dùng `await`
- Không kiểm tra `null`
- Gọi picker khi widget đã `dispose`
- Dùng context sai cấp

III. TIME PICKER TRONG FLUTTER

◈ showTimePicker là gì?
- Mở dialog chọn giờ
- Trả về `Future<TimeOfDay?>`

VS TimeOfDay vs DateTime
- `TimeOfDay`
  - chỉ có giờ + phút
  - **không có ngày**
- `DateTime`
  - có ngày + giờ đầy đủ

👉 Flutter tách riêng để:
- tránh nhầm lẫn logic
- dễ dùng trong form

🔗 Kết hợp Date + Time trong app
- Chọn ngày → `DateTime`
- Chọn giờ → `TimeOfDay`
- Ghép lại → `DateTime(year, month, day, hour, minute)`

❌ Sai lầm thường gặp
- Convert sai giờ (AM/PM)
- Quên ghép date + time
- Nghĩ TimeOfDay là DateTime

📱 Liên hệ form thật
- Đặt lịch họp

PhucPT10@fpt.edu.vn

- Nhắc việc
- Booking dịch vụ

🔷 PHẦN B. THEMING & UI POLISHING

(Trải nghiệm người dùng & tính chuyên nghiệp)

IV. TƯ DUY UI NHẤT QUÁN TRONG FLUTTER

❓ Vì sao UI "chạy được" chưa đủ?

- Người dùng **đánh giá app bằng mắt trước**
- UI rối → app bị xem là:
  - thiếu chuyên nghiệp
  - khó dùng

🔑 3 khái niệm cốt lõi

- **Consistency** – nhất quán
- **Hierarchy** – phân cấp thông tin
- **Readability** – dễ đọc

🔍 So sánh

- ❌ Không dùng Theme
  → mỗi màn hình một kiểu
- ✅ Có Theme
  → app có "bản sắc"

V. THEME DATA & MATERIAL THEME

🔷 ThemeData là gì?

- Là **cấu hình giao diện dùng chung**
- Được khai báo trong `MaterialApp`

🔷 Vai trò của `MaterialApp.theme`

- Áp dụng theme cho **toàn bộ widget con**
- Giúp:
  - code gọn
  - UI đồng bộ

🔑 Thành phần quan trọng

- `colorScheme`
- `textTheme`
- `buttonTheme` (mức cơ bản)

❌ Lỗi phổ biến

- Hard-code màu
- Chỉnh UI từng widget
- Không dùng theme từ đầu → khó sửa về sau

VI. SỬ DỤNG THEME TRONG UI

🔷 Theme.of(context)

- Lấy theme **gần nhất trong widget tree**

🧠 Context là gì (ở mức này)?

- Là **vị trí của widget trong cây UI**

PhucPT10@fpt.edu.vn

- Context quyết định:
  - theme nào được áp dụng
  - navigator nào được dùng

❌ Lỗi thường gặp

- Dùng context sai cấp
- Gọi `Theme.of(context)` trước `MaterialApp`
- Không hiểu vì sao theme không áp dụng

🔷 PHẦN C. NAVIGATOR 1.0 – QUẢN LÝ LUỒNG MÀN HÌNH

VII. TƯ DUY ĐIỀU HƯỚNG TRONG APP FLUTTER

❗ App nhiều màn hình ≠ nhiều widget

- Mỗi màn hình = **1 route**
- Navigator quản lý các route

🔑 Khái niệm cốt lõi

- **Route**: một màn hình
- **Stack navigation**:
  - vào → push
  - quay lại → pop

🔍 Liên hệ

- Android: back stack
- iOS: navigation stack
- Web: history

VIII. NAVIGATOR PUSH & POP

🔷 Navigator.push

- Thêm màn hình mới vào stack

🔷 Navigator.pop

- Loại màn hình hiện tại khỏi stack

🔁 Luồng ví dụ

- Home
  - push → Create
  - pop → Home

👉 Stack giúp Flutter:

- nhớ màn hình trước
- quay lại đúng chỗ

IX. TRUYỀN DỮ LIỆU QUA NAVIGATION (CƠ BẢN)

🔷 Khi push

- Truyền dữ liệu qua constructor

🔷 Khi pop

- Trả dữ liệu về màn trước

⚖️ Khi nào nên dùng?

- App nhỏ – vừa
- Dữ liệu đơn giản

👉 Chuẩn bị tư duy cho:

PhucPT10@fpt.edu.vn

- state management
- navigation nâng cao

X. LỖI THƯỜNG GẶP VỚI NAVIGATOR

❌ Push lặp màn hình

- Nguyên nhân: không kiểm soát luồng
- Cách tránh: vẽ flow trước khi code

❌ Pop sai context

- Dấu hiệu: pop không hoạt động
- Nguyên nhân: context không thuộc Navigator

❌ Mất dữ liệu khi quay lại

- Nguyên nhân: không truyền result
- Giải pháp: `pop(context, result)`

❌ UI không cập nhật

- Nguyên nhân: không setState
- Giải pháp: xử lý dữ liệu sau pop

XI. TỔNG KẾT & ĐỊNH HƯỚNG SLOT 7

🔑 Vai trò từng phần

- **Pickers** → UX nhập liệu đúng
- **Theme** → app chuyên nghiệp
- **Navigator** → luồng rõ ràng

🚀 Chuẩn bị cho các slot sau

- Form nâng cao
- Navigation 2.0
- State management
- App nhiều màn hình thật

PhucPT10@fpt.edu.vn

V. KINH NGHIỆM THỰC TIỄN, TIPS

1. KỸ THUẬT: DATE PICKER (`showDatePicker`)

Kinh nghiệm thực tế khi dùng trong dự án

- Dùng trong:
  - form tạo task (deadline)
  - app booking (ngày đặt)
  - hồ sơ cá nhân (ngày sinh)
- Ngoài ví dụ học thuật, **vấn đề thật nằm ở UX & validate**, không phải cú pháp

Tình huống thực tế

- **Bối cảnh**: App quản lý công việc cho sinh viên
- **Vấn đề**:
  - nhập deadline bằng TextField → sai định dạng
  - backend reject data
- **Vì sao chọn DatePicker?**
  - UX chuẩn
  - dữ liệu có cấu trúc
  - giảm validate thủ công

Lỗi thường gặp

Lỗi 1: Picker không mở

- Nguyên nhân:
  - dùng `context` đã bị dispose
  - gọi trong `initState`
- Ví dụ:
  - gọi picker ngay khi build widget
- Khắc phục:
  - chỉ gọi picker trong `onPressed` / `onTap`
- Phòng tránh:
  - picker = **phản hồi hành động người dùng**

Lỗi 2: App crash khi đóng picker

- Nguyên nhân:
  - không check `null`
- Ví dụ:
- `final date = await showDatePicker(...);`
- `print(date!.day); // crash nếu user cancel`
- Khắc phục:
  - luôn kiểm tra `if (date != null)`
- Phòng tránh:
  - nhớ: **user có quyền không chọn**

Tips & Tricks

- Luôn set `firstDate` / `lastDate` hợp lý
- Format ngày ngay sau khi chọn
- Log giá trị date khi debug UX

Best Practices

- ❌ Không dùng TextField cho ngày
- ✅ Picker + hiển thị kết quả

PhucPT10@fpt.edu.vn

- ❌ Không xử lý logic trong UI
- ✅ Tách hàm xử lý date

🎓 Bài học rút ra

Picker là **giải pháp UX**, không phải widget cho vui
Nếu chỉ nhớ 1 điều: **đừng cho người dùng nhập ngày bằng tay**

🔷 2. KỸ THUẬT: TIME PICKER (`showTimePicker`)

✅ Kinh nghiệm thực tế

- Dùng trong:
  - app nhắc việc
  - lịch học
  - booking giờ dịch vụ
- Vấn đề thật nằm ở **convert & hiển thị giờ**

💡 Tình huống thực tế

- **Bối cảnh**: App nhắc việc
- **Vấn đề**:
  - người dùng nhập "7:00"
  - không rõ sáng hay tối
- **Vì sao dùng TimePicker?**
  - trực quan
  - tránh nhập sai

⚠️ Lỗi thường gặp

❌ *Lỗi: Convert TimeOfDay sai*

- 🔍 Nguyên nhân:
  - nhầm `TimeOfDay` là `DateTime`
- 🧪 Tình huống:
  - lưu giờ nhưng quên ghép với ngày
- 🛠 Khắc phục:
  - luôn ghép `Date + Time` thành `DateTime`
- 🚫 Phòng tránh:
  - hiểu rõ **TimeOfDay chỉ là giờ**

💡 Tips & Tricks

- Hiển thị giờ theo format quen thuộc (HH:mm)
- Test cả AM / PM
- Log giờ khi debug

🛡 Best Practices

- Không lưu `TimeOfDay` riêng lẻ cho business logic
- Chỉ dùng nó cho UI chọn giờ

🎓 Bài học rút ra

TimePicker giúp **tránh nhầm giờ**, nhưng lập trình viên phải ghép đúng dữ liệu

🔷 3. KỸ THUẬT: THEME DATA & MATERIAL THEME

✅ Kinh nghiệm thực tế

- Trong dự án thật:

PhucPT10@fpt.edu.vn

    - 1 app = 1 phong cách
    - UI không được "mỗi màn hình một màu"

💡 Tình huống thực tế

- **Bối cảnh**: App nội bộ công ty
- **Vấn đề**:
    - mỗi dev set màu riêng
    - UI rối, khó review
- **Vì sao dùng ThemeData?**
    - kiểm soát tập trung
    - sửa 1 chỗ → đổi toàn app

⚠️ Lỗi thường gặp

❌ *Hard-code màu khắp nơi*

- 🔍 Nguyên nhân:
    - tư duy "làm cho xong"
- 🧪 Dấu hiệu:
    - code UI dài, khó đọc
- 🛠 Khắc phục:
    - đưa màu vào Theme
- 🚫 Phòng tránh:
    - **luôn tạo theme từ đầu**

💡 Tips & Tricks

- Dùng ít màu (2–3)
- Đặt tên màu theo vai trò, không theo màu sắc

🛡 Best Practices

- ❌ Không chỉnh màu từng widget
- ✅ Theme chung
- ❌ Không để UI quyết định style
- ✅ Theme quyết định

🎓 Bài học rút ra

Theme giúp app **nhìn chuyên nghiệp**, dù logic đơn giản

🔷 4. KỸ THUẬT: SỬ DỤNG THEME TRONG UI (`Theme.of(context)`)

⚠️ Lỗi thường gặp

❌ *Theme không áp dụng*

- 🔍 Nguyên nhân:
    - dùng context sai cấp
- 🛠 Khắc phục:
    - đảm bảo widget nằm dưới `MaterialApp`
- 🚫 Phòng tránh:
    - hiểu context là **vị trí trong cây UI**

🎓 Bài học rút ra

Context quyết định **bạn đang dùng theme nào**

PhucPT10@fpt.edu.vn

5. KỸ THUẬT: NAVIGATOR 1.0 (`push` / `pop`)

Kinh nghiệm thực tế

- App thật:
  - luôn có luồng rõ
  - người dùng phải biết "đang ở đâu"

Tình huống thực tế

- **Bối cảnh**: Todo App
- **Vấn đề**:
  - push liên tục
  - back không đúng
- **Vì sao dùng Navigator đúng chuẩn?**
  - kiểm soát stack
  - UX dễ hiểu

Lỗi thường gặp

Push lặp màn hình

- Nguyên nhân:
  - không vẽ flow
- Khắc phục:
  - xác định rõ Home → Detail → Back
- Phòng tránh:
  - **vẽ sơ đồ trước khi code**

Pop không hoạt động

- Nguyên nhân:
  - sai context
- Khắc phục:
  - dùng context thuộc Navigator

Tips & Tricks

- Test back liên tục
- Log khi push/pop
- Không push nếu chỉ cần cập nhật UI

Best Practices

- Push khi sang màn mới
- Pop khi quay lại
- Không lạm dụng truyền dữ liệu lớn qua Navigator

Bài học rút ra

Navigation là **luồng sử dụng**, không phải trò chơi push/pop

TỔNG KẾT BÀI HỌC SLOT 7

Nếu sinh viên chỉ nhớ **3 điều này**, là đủ nền tảng:

1. **Picker = UX chuẩn**, không cho nhập tay
2. **Theme = app chuyên nghiệp**, không hard-code
3. **Navigation = luồng người dùng**, không push bừa

PhucPT10@fpt.edu.vn

EXERCISE

BÀI TẬP TỔNG HỢP 1

Pickers – Theming – Navigator 1.0

1 TỔNG QUAN DỰ ÁN

Tên dự án

Simple Planner App

Mô tả ngắn gọn

Ứng dụng Flutter đa màn hình cho phép người dùng:

- Xem danh sách kế hoạch đơn giản
- Tạo kế hoạch mới bằng cách **chọn ngày & giờ**
- Di chuyển giữa các màn hình bằng **Navigator**
- Giao diện **nhất quán nhờ Theme**

👉 Đây là dạng app **rất gần với app thật** (planner, todo, booking mini).

Mục tiêu học tập

Sau dự án này, sinh viên **bắt buộc làm được**:

- Dùng **DatePicker / TimePicker đúng ngữ cảnh UX**
- Hiểu và dùng **Navigator.push / pop** đúng luồng
- Áp dụng **ThemeData cho toàn app**, không hard-code UI
- Nhìn app dưới góc độ **người dùng đang đi qua các màn hình**

Đối tượng phù hợp

- Sinh viên vừa học xong Slot 7
- Trình độ trung bình – yếu
- Cần **củng cố tư duy app đa màn hình**

2 CHỨC NĂNG CHÍNH CỦA ỨNG DỤNG

Màn hình Home

- Hiển thị danh sách kế hoạch (tạm thời có thể là 1 item)
- Có nút **"Tạo kế hoạch mới"**

👉 Gắn với:

- **Navigator.push**
- **Theme (AppBar, Button)**

➕ Màn hình Create Plan

- Nhập tiêu đề kế hoạch
- Chọn **Date**
- Chọn **Time**
- Bấm **Save** để quay lại Home

👉 Gắn với:

- **showDatePicker**
- **showTimePicker**
- **Navigator.pop (trả dữ liệu)**

Navigation Flow

```
Home Screen
   ↓ (push)
Create Plan Screen
   ↓ (pop + data)
Home Screen (cập nhật UI)
```

📌 Sinh viên phải luôn trả lời được:

PhucPT10@fpt.edu.vn

“Người dùng đang ở màn hình nào? Bấm xong thì đi đâu?”

3 CÔNG NGHỆ & CÔNG CỤ

- Flutter SDK (Material Design)
- Dart
- Android Studio / VS Code
- Emulator (Android / iOS)

🚫 Không dùng:

- Provider / Bloc
- Navigator 2.0
- Kiến trúc phức tạp

4 CẤU TRÚC THƯ MỤC DỰ ÁN (RẤT QUAN TRỌNG)

```
lib/
├── main.dart
├── theme/
│   └── app_theme.dart
├── screens/
│   ├── home_screen.dart
│   └── create_plan_screen.dart
├── widgets/
│   └── primary_button.dart
└── models/
    └── plan.dart
```

🧠 Vì sao phải tách như vậy?

- `theme/`
  → UI dùng chung, **sửa 1 chỗ – đổi toàn app**
- `screens/`
  → Mỗi màn hình là **1 route – 1 file**
- `widgets/`
  → Tránh lặp code, UI gọn gàng

📌 **Tư duy quan trọng:**
App nhiều màn hình ≠ nhét hết vào `main.dart`

5 DANH SÁCH CÁC BƯỚC THỰC HIỆN (TÓM TẮT)

1. Tạo project Flutter mới
2. Tạo `ThemeData` cho toàn app
3. Áp dụng Theme vào `MaterialApp`
4. Tạo `HomeScreen`
5. Thêm nút điều hướng sang Create Plan
6. Tạo `CreatePlanScreen`
7. Thêm TextField nhập tiêu đề
8. Thêm DatePicker
9. Thêm TimePicker
10. Trả dữ liệu về Home
11. Hiển thị dữ liệu lên UI
12. Test toàn bộ luồng người dùng

6 HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC

◈ BƯỚC 1: Setup project & chạy thử

Mục tiêu:
Đảm bảo môi trường chạy OK.
Thao tác:

PhucPT10@fpt.edu.vn

```
flutter create simple_planner
cd simple_planner
flutter run
```

✅ Kiểm tra:

- App chạy được
- Không lỗi đỏ

⚠️ Lỗi hay gặp:

- Chưa bật emulator
- Flutter SDK chưa đúng version

◈ BƯỚC 2: Tạo ThemeData

📌 **Mục tiêu:**

Hiểu: *Theme là nền tảng UI, không phải trang trí.*

File: `lib/theme/app_theme.dart`

```
import 'package:flutter/material.dart';

class AppTheme {
  static ThemeData lightTheme = ThemeData(
    useMaterial3: true,
    colorScheme: ColorScheme.fromSeed(
      seedColor: Colors.blue,
      brightness: Brightness.light,
    ),
    appBarTheme: const AppBarTheme(
      centerTitle: true,
    ),
    inputDecorationTheme: InputDecorationTheme(
      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(12),
      ),
    ),
  );
}
```

💡 Tip:

- Dùng `ColorScheme.fromSeed` để UI đồng bộ màu

◈ BƯỚC 3: Áp dụng Theme vào MaterialApp

File: `main.dart`

```
import 'package:flutter/material.dart';
import 'screens/home_screen.dart';
import 'theme/app_theme.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: AppTheme.lightTheme,
```


PhucPT10@fpt.edu.vn

```
      home: const HomeScreen(),
    );
  }
}
```

✅ Kiểm tra:

- AppBar, button có màu theo theme

⚠️ Lỗi thường gặp:

- Khai báo Theme nhưng **quên gán cho MaterialApp**

4) BƯỚC 4 — Tạo model Plan (dữ liệu kế hoạch)

Mục đích

- Khi `pop(data)` về Home, ta trả về một object rõ ràng.

lib/models/plan.dart

```
import 'package:flutter/material.dart';

class Plan {
  final String title;
  final DateTime date; // chỉ lấy phần ngày
  final TimeOfDay time;

  const Plan({
    required this.title,
    required this.date,
    required this.time,
  });
}
```

5) BƯỚC 5 — Tạo PrimaryButton (tránh lặp UI)

Mục đích

- Tránh viết `ElevatedButton(...)` lặp lại nhiều lần.
- Dạy sinh viên thói quen "widget dùng chung".

lib/widgets/primary_button.dart

```
import 'package:flutter/material.dart';

class PrimaryButton extends StatelessWidget {
  final String text;
  final VoidCallback? onPressed;

  const PrimaryButton({
    super.key,
    required this.text,
    required this.onPressed,
  });

  @override
  Widget build(BuildContext context) {
    return SizedBox(
      width: double.infinity,
      height: 48,
      child: ElevatedButton(
```


PhucPT10@fpt.edu.vn

```
        onPressed: onPressed,
        child: Text(text),
      ),
    );
  }
}
```

Kiểm chứng
Chưa dùng thì chưa thấy thay đổi, bình thường.

6) BƯỚC 6 — HomeScreen: hiển thị danh sách + nút điều hướng
Mục đích

- Home là màn hình gốc (route đầu).
- Khi bấm "Create Plan" → `Navigator.push(...)`.
- Khi CreatePlan `pop(plan)` → Home nhận plan và cập nhật UI bằng `setState`.

Lưu ý quan trọng cho sinh viên yếu:
Home phải là `StatefulWidget` vì có dữ liệu thay đổi (list).
lib/screens/home_screen.dart

```
import 'package:flutter/material.dart';
import '../models/plan.dart';
import '../widgets/primary_button.dart';
import 'create_plan_screen.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  final List<Plan> _plans = [];

  Future<void> _goToCreatePlan(BuildContext context) async {
    // 1) push sang màn hình tạo plan
    final result = await Navigator.push<Plan>(
      context,
      MaterialPageRoute(builder: (_) => const CreatePlanScreen()),
    );

    // 2) result có thể null nếu người dùng bấm back hoặc cancel
    if (result == null) return;

    // 3) có data -> cập nhật UI
    setState(() {
      _plans.insert(0, result);
    });
  }

  String _formatDate(DateTime date) {
    // format đơn giản: YYYY-MM-DD
```


PhucPT10@fpt.edu.vn

```
    final y = date.year.toString().padLeft(4, '0');
    final m = date.month.toString().padLeft(2, '0');
    final d = date.day.toString().padLeft(2, '0');
    return "$y-$m-$d";
  }

  String _formatTime(TimeOfDay time) {
    final hh = time.hour.toString().padLeft(2, '0');
    final mm = time.minute.toString().padLeft(2, '0');
    return "$hh:$mm";
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('My Planner')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            PrimaryButton(
              text: 'Create Plan',
              onPressed: () => _goToCreatePlan(context),
            ),
            const SizedBox(height: 16),
            Expanded(
              child: _plans.isEmpty
                  ? const Center(
                      child: Text(
                        'No plans yet.\nTap "Create Plan" to add one.',
                        textAlign: TextAlign.center,
                      ),
                    )
                  : ListView.separated(
                      itemCount: _plans.length,
                      separatorBuilder: (_, __) => const SizedBox(height:
12),
                      itemBuilder: (context, index) {
                        final plan = _plans[index];
                        return Card(
                          child: ListTile(
                            title: Text(plan.title),
                            subtitle: Text(
                              '${_formatDate(plan.date)} •
${_formatTime(plan.time)}',
                            ),
                          ),
                        );
                      },
                    ),
            ),
          ],
        ),
```


PhucPT10@fpt.edu.vn

```
      ),
    );
  }
}
```

Kiểm chứng bước này

Chạy app:

- Thấy AppBar "My Planner".
- Có nút "Create Plan".
- Dưới có text "No plans yet..."

Bấm nút:

- Nếu chưa có CreatePlanScreen thì báo lỗi import → làm bước tiếp theo.

7) BƯỚC 7 — CreatePlanScreen: nhập title + chọn date/time + save

Mục đích UX (giải thích cho sinh viên)

- Không để user tự gõ ngày giờ (dễ sai) → dùng picker.
- Picker là **async** (phải `await`).
- Người dùng có thể bấm Cancel → trả `null` → phải xử lý.

lib/screens/create_plan_screen.dart

```
import 'package:flutter/material.dart';
import '../models/plan.dart';
import '../widgets/primary_button.dart';

class CreatePlanScreen extends StatefulWidget {
  const CreatePlanScreen({super.key});

  @override
  State<CreatePlanScreen> createState() => _CreatePlanScreenState();
}

class _CreatePlanScreenState extends State<CreatePlanScreen> {
  final _titleController = TextEditingController();

  DateTime? _selectedDate;
  TimeOfDay? _selectedTime;

  @override
  void dispose() {
    _titleController.dispose();
    super.dispose();
  }

  Future<void> _pickDate() async {
    final now = DateTime.now();

    final picked = await showDatePicker(
      context: context,
      initialDate: _selectedDate ?? now,
      firstDate: DateTime(now.year - 1),
      lastDate: DateTime(now.year + 2),
      helpText: 'Select plan date',
    );
```


PhucPT10@fpt.edu.vn

```
    if (picked == null) return; // user cancel

    setState(() {
      // lưu đúng phần ngày (năm-tháng-ngày)
      _selectedDate = DateTime(picked.year, picked.month, picked.day);
    });
  }

  Future<void> _pickTime() async {
    final picked = await showTimePicker(
      context: context,
      initialTime: _selectedTime ?? TimeOfDay.now(),
      helpText: 'Select plan time',
    );

    if (picked == null) return; // user cancel

    setState(() {
      _selectedTime = picked;
    });
  }

  String _dateLabel() {
    if (_selectedDate == null) return 'Choose Date';
    final d = _selectedDate!;
    final y = d.year.toString().padLeft(4, '0');
    final m = d.month.toString().padLeft(2, '0');
    final day = d.day.toString().padLeft(2, '0');
    return '$y-$m-$day';
  }

  String _timeLabel() {
    if (_selectedTime == null) return 'Choose Time';
    final t = _selectedTime!;
    final hh = t.hour.toString().padLeft(2, '0');
    final mm = t.minute.toString().padLeft(2, '0');
    return '$hh:$mm';
  }

  void _save() {
    final title = _titleController.text.trim();

    // Validate đơn giản (đúng yêu cầu sinh viên yếu)
    if (title.isEmpty) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Title is required')),
      );
      return;
    }
    if (_selectedDate == null) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Please choose a date')),
```


PhucPT10@fpt.edu.vn

```
      );
      return;
    }
    if (_selectedTime == null) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Please choose a time')),
      );
      return;
    }

    final plan = Plan(
      title: title,
      date: _selectedDate!,
      time: _selectedTime!,
    );

    // pop không chỉ quay lại, mà trả dữ liệu
    Navigator.pop(context, plan);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Create Plan')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _titleController,
              decoration: const InputDecoration(
                labelText: 'Plan Title',
                hintText: 'e.g. Study Flutter',
              ),
              textInputAction: TextInputAction.done,
            ),
            const SizedBox(height: 16),
            Row(
              children: [
                Expanded(
                  child: OutlinedButton(
                    onPressed: _pickDate,
                    child: Text(_dateLabel()),
                  ),
                ),
                const SizedBox(width: 12),
                Expanded(
                  child: OutlinedButton(
                    onPressed: _pickTime,
                    child: Text(_timeLabel()),
                  ),
                ),
              ],
```


PhucPT10@fpt.edu.vn

```
            ),
            const Spacer(),
            PrimaryButton(
              text: 'Save',
              onPressed: _save,
            ),
          ],
        ),
      ),
    );
  }
}
```

Kiểm chứng CreatePlanScreen

1. Bấm **Create Plan** từ Home → thấy màn hình "Create Plan".
2. Bấm **Choose Date** → DatePicker mở.
3. Bấm **Choose Time** → TimePicker mở.
4. Nếu bấm Save khi thiếu dữ liệu → SnackBar báo lỗi.
5. Điền đủ → Save → quay về Home và thấy item mới xuất hiện.

8) Checklist "đúng kiến thức" (bắt sinh viên trả lời)

A. Picker

- Vì sao `showDatePicker` phải `await`?
  → vì user đang chọn trên UI, phải chờ kết quả trả về.
- Vì sao phải xử lý `null`?
  → user có thể cancel.

B. Navigator

- `push` nghĩa là gì?
  → thêm màn hình lên "stack".
- pop nghĩa là gì?
  → bỏ màn hình hiện tại khỏi stack và quay về màn hình trước.
- `pop(context, data)` nghĩa là gì?
  → quay lại + **trả kết quả** cho màn hình trước.

C. State

- Vì sao Home là `StatefulWidget`?
  → vì danh sách `_plans` thay đổi nên phải `setState`.

7 KIỂM TRA & CHẠY ỨNG DỤNG

- `flutter run`
- Thử:
  - Chuyển màn hình
  - Chọn date/time
  - Quay lại Home

✅ Quan sát:

- Luồng có hợp lý không?
- Người dùng có bị rối không?

PhucPT10@fpt.edu.vn

8 CÁC LỖI PHỔ BIẾN & CÁCH XỬ LÝ

Lỗi	Nguyên nhân
Picker không mở	Context sai
UI không cập nhật	Quên setState
Theme không áp dụng	Đặt sai cấp
Push mãi không pop	Không hiểu stack

9 KẾT QUẢ ĐẠT ĐƯỢC SAU DỰ ÁN

Sinh viên **hiểu thật sự**:

- Navigation là **UX flow**
- Theme là **nền tảng UI**
- Picker giúp **tránh sai dữ liệu**

🎯 Sẵn sàng học:

- Navigation nâng cao
- State management
- App nhiều màn hình thật

10 BÀI TẬP MỞ RỘNG (KHUYẾN NGHỊ)

- Thêm màn hình Confirm
- Thêm Dark Theme
- Validate date/time
- Cải thiện UX khi chưa chọn đủ dữ liệu

PhucPT10@fpt.edu.vn

TỪ ĐIỂN THUẬT NGỮ (FOUNDATION GLOSSARY)

NHÓM 1 – USER INTERACTION & PICKERS

Thuật ngữ	Giải thích dễ hiểu	Dễ hiểu sai ở đâu?	Liên hệ app thật
User Interaction	Cách người dùng **tác động vào app** (bấm, chọn, nhập, vuốt…)	Nghĩ chỉ là onPressed	Mọi hành vi người dùng đều là interaction
Picker	UI đặc biệt giúp người dùng **chọn dữ liệu đúng chuẩn**	Nghĩ là TextField nâng cao	Chọn ngày, giờ, số lượng
DatePicker	Giao diện chọn **ngày tháng** do hệ thống cung cấp	Nghĩ là widget gắn trực tiếp vào UI	Mở ra dạng dialog
TimePicker	Giao diện chọn **giờ phút**	Nhầm với DateTime	Trả về TimeOfDay
showDatePicker	Hàm **mở DatePicker**	Nghĩ chạy đồng bộ	Luôn là async
showTimePicker	Hàm **mở TimePicker**	Quên xử lý null	Người dùng có thể bấm Cancel
async / await	Cách Flutter **chờ người dùng thao tác xong**	Nghĩ là chạy chậm	Thực ra là chờ tương tác
null result	Không có giá trị trả về (user hủy)	Không check null → crash	UX rất hay gặp
DateTime	Kiểu dữ liệu chứa ngày + giờ	Dùng sai format	Dùng cho lưu trữ
TimeOfDay	Chỉ chứa giờ + phút	Nhầm là DateTime	Phải convert khi lưu

📌 **Ghi nhớ cho sinh viên:**
Picker không phải để "cho đẹp", mà để **tránh sai dữ liệu & UX kém**.

NHÓM 2 – THEMING & UI POLISHING

Thuật ngữ	Giải thích dễ hiểu	Dễ hiểu sai ở đâu?	Liên hệ app thật
UI Polishing	Làm UI **dễ nhìn – dễ dùng – chuyên nghiệp hơn**	Nghĩ là trang trí	UX quyết định giữ người dùng
Theme	Bộ quy tắc **màu – chữ – style dùng chung toàn app**	Nghĩ chỉ là màu	Ảnh hưởng toàn bộ UI
ThemeData	Nơi **định nghĩa Theme**	Khai báo muộn	Nên làm từ đầu
MaterialApp.theme	Gán Theme cho toàn app	Nhầm với widget theme	Là gốc của UI
ColorScheme	Bộ màu có vai trò rõ ràng	Hard-code màu	Dễ rebrand
TextTheme	Quy định style chữ theo cấp bậc	Font loạn	Đọc rất mệt
Consistency	UI nhất quán giữa các màn hình	Mỗi màn hình 1 kiểu	App nhìn "nghiệp dư"
Hierarchy	Thứ bậc thị giác	Mọi thứ nổi như nhau	Người dùng không biết nhìn đâu
Readability	Mức độ dễ đọc, dễ hiểu	Chữ nhỏ, màu kém	UX kém nhưng không báo lỗi
Hard-code UI	Ghi trực tiếp màu, padding	Nhanh ban đầu	Rất khó sửa sau này

📌 **Ghi nhớ cho sinh viên:**
UI "chạy được" ≠ UI "dùng được".

NHÓM 3 – NAVIGATION & LUỒNG MÀN HÌNH

Thuật ngữ	Giải thích dễ hiểu	Dễ hiểu sai ở đâu?	Liên hệ app thật
Navigation	Cách người dùng **di chuyển giữa các màn hình**	Nghĩ là mở widget	Là luồng UX

PhucPT10@fpt.edu.vn

Route	Một màn hình trong app	Nhầm với widget nhỏ	Screen-level
Navigator	Bộ quản lý **các màn hình**	Dùng bừa context	Dễ bug
Navigator 1.0	Điều hướng dạng **stack**	Không hiểu stack	Home → Detail → Back
push	Mở màn hình mới	Push trùng màn hình	Stack phình to
pop	Quay lại màn hình trước	Pop sai context	Crash hoặc không quay
Stack navigation	Màn hình xếp chồng như chồng sách	Không hình dung được	iOS / Android truyền thống
MaterialPageRoute	Cách tạo route kiểu Material	Copy-paste	Nên hiểu bản chất
Passing data	Truyền dữ liệu khi chuyển màn hình	Truyền quá nhiều	Coupling cao
Context	Vị trí widget trong cây UI	Dùng sai	Lỗi rất khó debug
Replace screen	Thay màn hình hiện tại	Nhầm với push	UX khác hoàn toàn
Flow	Chuỗi màn hình người dùng đi qua	Không thiết kế trước	UX rối

📌 **Ghi nhớ cho sinh viên:**
Navigation sai → UX sai → app bị bỏ.

🧠 NHÓM 4 – TƯ DUY DỰ ÁN & HỌC THUẬT

Thuật ngữ	Giải thích dễ hiểu	Dễ hiểu sai ở đâu?	Liên hệ thực tế
UX (User Experience)	Cảm nhận khi dùng app	Chỉ nghĩ UI	UX là hành vi
Maintainability	Dễ sửa – dễ nâng cấp	Không quan tâm	Rất đau sau 6 tháng
Scalability	Khả năng mở rộng app	Nghĩ để sau	Quyết định từ sớm
Consistency UI	UI giống nhau có chủ đích	Nhàm chán	Thực ra là chuyên nghiệp
Trade-off	Đánh đổi kỹ thuật	Muốn mọi thứ	Không tồn tại giải pháp hoàn hảo
Tech Debt	Nợ kỹ thuật	Không thấy ngay	Bùng nổ về sau
Demo vs Product	App học vs app thật	Không phân biệt	Slot 7 là ranh giới

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU CHUYÊN SÂU

1 DatePicker / TimePicker & bản chất UX

Phân tích vai trò của DatePicker và TimePicker trong trải nghiệm người dùng (UX) thực tế.
Vì sao Flutter **không cho** sử dụng Picker như một widget thông thường trong widget tree, mà yêu cầu gọi thông qua `showDatePicker()` / `showTimePicker()`?
Thiết kế này giúp Flutter giải quyết vấn đề UX và kỹ thuật nào?

👉 Gợi ý tư duy:

- Modal vs inline UI
- Focus của người dùng
- Kiểm soát context & platform behavior

2 Picker vs TextField – đúng ngữ cảnh hay không?

So sánh việc chọn ngày/giờ bằng Picker với việc nhập thủ công bằng `TextField`.
Trong những trường hợp nào **Picker là bắt buộc** để đảm bảo UX và tính đúng dữ liệu?
Khi nào việc dùng Picker lại gây ra trải nghiệm kém hoặc làm chậm thao tác của người dùng?

👉 Gợi ý tư duy:

- Loại người dùng (end-user vs admin)
- Tần suất thao tác
- Rủi ro nhập sai dữ liệu

3 ThemeData đặt ở đâu là "đúng kiến trúc"?

Giải thích vì sao `ThemeData` nên được khai báo ở `MaterialApp` thay vì cấu hình màu sắc, font chữ trong từng màn hình riêng lẻ.
Nếu mỗi screen tự định nghĩa style, điều gì sẽ xảy ra khi dự án mở rộng từ 3 màn hình lên 20 màn hình?

👉 Gợi ý tư duy:

- Single source of truth
- UI consistency
- Chi phí bảo trì (maintenance cost)

4 Theming & khả năng mở rộng ứng dụng

Phân tích mối quan hệ giữa Theming và khả năng mở rộng (scalability) của ứng dụng.
Một app **không dùng Theme ngay từ đầu** sẽ gặp những khó khăn gì khi cần:

- đổi màu thương hiệu
- hỗ trợ Dark Mode
- tái sử dụng UI component?

👉 Gợi ý tư duy:

- Technical debt
- Refactor cost
- UX consistency theo thời gian

5 Navigator 1.0 & tư duy "luồng người dùng"

Đánh giá vai trò của Navigator 1.0 trong việc hình thành tư duy **User Flow**.
Vì sao việc học Navigation sớm giúp sinh viên viết UI tốt hơn, ngay cả khi app còn rất nhỏ và chưa có backend?

👉 Gợi ý tư duy:

- Người dùng đang ở đâu?
- Đi đâu tiếp theo?
- Trở lại bằng cách nào?

PhucPT10@fpt.edu.vn

6 push/pop vs if/else ẩn–hiện widget

So sánh việc điều hướng màn hình bằng `Navigator.push/pop` với cách "ẩn/hiện widget" bằng `if/else` hoặc `setState`.
Vì sao cách ẩn/hiện widget **không thể thay thế Navigation** trong các ứng dụng thực tế?

👉 Gợi ý tư duy:

- Back stack
- Android back button
- Trạng thái lịch sử (history)

7 Lạm dụng Navigator.push – hậu quả dài hạn

Phân tích rủi ro khi liên tục sử dụng `Navigator.push()` mà không kiểm soát `pop()`.
Điều gì sẽ xảy ra với:

- bộ nhớ (memory)
- trải nghiệm người dùng
- hành vi nút Back
  khi navigation stack ngày càng phình to?

👉 Gợi ý tư duy:

- Stack growth
- Người dùng bị "lạc"
- App ngày càng khó debug

8 Vì sao Picker – Theme – Navigation luôn đi cùng nhau?

Giải thích vì sao trong các ứng dụng thực tế, Pickers – Theme – Navigation thường xuất hiện cùng lúc.
Ba kỹ thuật này kết hợp với nhau để giải quyết **nhóm vấn đề UX nào** mà nếu thiếu một trong ba thì app sẽ trở nên rời rạc?

👉 Gợi ý tư duy:

- Trải nghiệm xuyên suốt
- Nhất quán UI
- Hành vi người dùng theo luồng

9 "UI chạy được" vs "UI có trải nghiệm tốt"

Đánh giá sự khác biệt giữa:

- một UI **chạy được**
- một UI **có trải nghiệm tốt**
  Trong sự chuyển đổi từ app demo sang app dùng thật, Pickers – Theme – Navigation đóng vai trò gì?

👉 Gợi ý tư duy:

- Demo cho lập trình viên
- Sản phẩm cho người dùng
- UX là yếu tố sống còn

10 Giới hạn của Navigator 1.0 khi app lớn dần

Dự đoán những giới hạn của Navigator 1.0 khi ứng dụng mở rộng về số màn hình và luồng phức tạp.
Những dấu hiệu nào cho thấy app đã đến lúc cần:

- Navigation nâng cao
- State management
- kiến trúc rõ ràng hơn?

👉 Gợi ý tư duy:

- Truyền dữ liệu phức tạp
- Điều hướng phụ thuộc trạng thái
- Khó kiểm soát luồng

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU THỰC TIỄN CHUYÊN NGHIỆP

1 Prompt 1 – Centralize Picker hay gọi trực tiếp?

Prompt cho sinh viên:
Trong dự án Flutter quy mô lớn, vì sao **DatePicker / TimePicker không nên được gọi trực tiếp ở mọi widget cần dùng**?
Hãy phân tích:

- Rủi ro về **UX consistency** nếu mỗi màn hình tự cấu hình picker
- Khó khăn về **maintain** khi thay đổi format, locale, rule ngày/giờ

Nếu bạn là Tech Lead:

- Bạn sẽ tổ chức picker ở mức nào (widget, service, helper, UI layer)?
- Đánh đổi của mỗi cách tiếp cận là gì?

🎯 Trọng tâm: **Consistency vs tốc độ code ban đầu**

2 Prompt 2 – Hard-code màu & font: cái giá của rebrand

Prompt cho sinh viên:
Đánh giá hậu quả của việc **hard-code màu sắc, font chữ** ở từng màn hình thay vì dùng `ThemeData`.
Phân tích kịch bản:

- Product cần **rebrand** (đổi màu chủ đạo, font)
- App đã có 20–30 màn hình

Trả lời:

- Chi phí sửa đổi là bao nhiêu?
- Những bug UI nào có thể phát sinh?
- Vì sao quyết định "cho nhanh" ban đầu lại trở thành gánh nặng?

🎯 Trọng tâm: **Maintainability & chi phí dài hạn**

3 Prompt 3 – Theme không thống nhất trong team

Prompt cho sinh viên:
Trong team nhiều developer, việc **không thống nhất Theme** có thể gây ra những vấn đề gì?
Phân tích:

- UI không đồng nhất ảnh hưởng thế nào đến perception của người dùng?
- Dev mới vào team sẽ gặp khó khăn gì?
- Tốc độ phát triển feature bị ảnh hưởng ra sao?

Nếu là Tech Lead:

- Bạn sẽ enforce Theme bằng cách nào?

🎯 Trọng tâm: **Team productivity & product quality**

4 Prompt 4 – Picker trong luồng nghiệp vụ thật

Prompt cho sinh viên:
Phân tích một luồng nghiệp vụ thật (ví dụ: đặt lịch khám, booking phòng, tạo task).
Hãy đánh giá:

- Date/Time Picker nên xuất hiện **ở thời điểm nào** trong flow?
- Nếu gọi picker quá sớm / quá muộn, UX bị ảnh hưởng ra sao?

Trả lời:

- Picker là input hay là **công cụ hỗ trợ quyết định**?
- Vì sao dùng sai thời điểm khiến người dùng bỏ flow?

🎯 Trọng tâm: **UX flow, không phải widget**

5 Prompt 5 – Navigation 1.0 khi app nhiều luồng

Prompt cho sinh viên:
Đánh giá việc dùng `Navigator.push / pop` trong app có:

- nhiều màn hình

PhucPT10@fpt.edu.vn

- nhiều luồng song song (onboarding, auth, main flow)

Phân tích:

- Stack navigation trở nên khó kiểm soát như thế nào?
- Khi nào Navigation 1.0 bắt đầu lộ giới hạn?

Kết luận:

- Dấu hiệu nào cho thấy app đã "quá lớn" cho Navigation 1.0 thuần túy?

🎯 Trọng tâm: **Giới hạn kiến trúc**

6 Prompt 6 – Truyền dữ liệu qua constructor: tiện hay nguy hiểm?

Prompt cho sinh viên:

Phân tích hậu quả của việc **truyền quá nhiều dữ liệu qua constructor khi navigation**.

Hãy đánh giá:

- Refactor màn hình sẽ khó thế nào?
- Màn hình bị phụ thuộc vào bao nhiêu nơi?

Nếu bạn phải maintain code này sau 6 tháng:

- Bạn sẽ gặp khó khăn gì?
- Khi nào nên dừng cách tiếp cận này?

🎯 Trọng tâm: **Coupling & refactor cost**

7 Prompt 7 – Navigation 1.0 vs state-driven navigation

Prompt cho sinh viên:

So sánh tư duy:

- Navigation dựa trên `push/pop`
- Navigation dựa trên state (ở mức khái niệm)

Trả lời:

- Vì sao app càng phức tạp càng khó quản lý bằng stack thuần?
- Navigation nâng cao giải quyết vấn đề gì mà Navigation 1.0 không làm được?

🎯 Trọng tâm: **Chuẩn bị tư duy cho slot sau**

8 Prompt 8 – Pop hay Replace: quyết định UX quan trọng

Prompt cho sinh viên:

Trong dự án thật:

- Khi nào nên `pop`?
- Khi nào nên `pushReplacement`?

Phân tích:

- Nếu dùng sai, người dùng có thể gặp:
  - quay lại màn hình không mong muốn
  - flow bị "lặp"

Hãy liên hệ với app thật (login, create flow).

🎯 Trọng tâm: **User journey & mental model**

9 Prompt 9 – UI Polishing & giá trị thương mại

Prompt cho sinh viên:

Đánh giá vai trò của:

- spacing
- typography
- color hierarchy

đối với sản phẩm thương mại.

Trả lời:

- Vì sao app "chạy đúng" nhưng vẫn bị đánh giá thấp?
- UX kém ảnh hưởng trực tiếp thế nào đến trust & retention?

🎯 Trọng tâm: **UI = giá trị sản phẩm**

PhucPT10@fpt.edu.vn

10 Prompt 10 – Theme & khả năng scale đa nền tảng

Prompt cho sinh viên:
Phân tích mối quan hệ giữa Theme và khả năng scale app sang:

- tablet
- desktop
- web

Nếu không chuẩn bị Theme từ sớm:

- Chi phí sửa UI sẽ tăng thế nào?
- Những phần nào phải đập đi làm lại?

🎯 Trọng tâm: **Scale cost**

11 Prompt 11 – Ai chịu trách nhiệm UX consistency?

Prompt cho sinh viên:
Trong team chuyên nghiệp:

- Ai chịu trách nhiệm cho Theme & UX consistency?

Phân tích:

- Vai trò của developer vs designer
- Dev nên làm gì để không "phá design"?

Nếu không có designer:

- Dev cần bù đắp bằng tư duy nào?

🎯 Trọng tâm: **Team collaboration**

12 Prompt 12 – UI logic viết trong widget lớn

Prompt cho sinh viên:
Phân tích rủi ro khi:

- navigation logic
- picker handling

bị viết trực tiếp trong widget lớn.
Đánh giá:

- Test khó ở điểm nào?
- Maintain gặp vấn đề gì?

Khi nào cần tách logic ra khỏi widget?
🎯 Trọng tâm: **Testability & clean architecture**

13 Prompt 13 – Slot 7 dưới góc nhìn kiến trúc

Prompt cho sinh viên:
Đánh giá Slot 7 (Picker – Theme – Navigation) dưới góc nhìn kiến trúc app.
Hãy trả lời:

- Những quyết định nào ở Slot 7 ảnh hưởng lâu dài?
- Quyết định nào khó sửa nhất khi app đã lớn?

🎯 Trọng tâm: **Early architectural decisions**

14 Prompt 14 – Nhiều form, nhiều picker, không lặp code

Prompt cho sinh viên:
Trong app có:

- nhiều form
- nhiều picker

Làm thế nào để:

- UX nhất quán
- không lặp code?

Phân tích hướng tiếp cận ở **mức tư duy**, chưa cần code nâng cao.

PhucPT10@fpt.edu.vn

🎯 Trọng tâm: **Abstraction & reuse**

1 5 Prompt 15 – Code review như Tech Lead

Prompt cho sinh viên:

Nếu bạn là Tech Lead, khi review code junior, bạn sẽ chú ý những điểm nào liên quan đến:

- Navigation
- Theme
- Picker?

Vì sao những điểm này quan trọng hơn cả việc "code chạy"?

🎯 Trọng tâm: **Professional mindset**

PhucPT10@fpt.edu.vn